

Data Visualization

Authors: Benedikt Prisett & Stijn Diemel

This notebook presents the visualizations that have been created to answer the provided questions. For each question we also include some of the earlier design iterations and then present the final visualization, that we ultimately used in the final multi-view dashboard.

Setup

```
In [1]: import altair as alt
import pandas as pd

alt.data_transformers.disable_max_rows()

df_q1 = pd.read_csv("data/clean_data/df_q1.csv")
df_q2 = pd.read_csv("data/clean_data/df_q2.csv")
df_q3 = pd.read_csv("data/clean_data/df_q3.csv")
df_q4 = pd.read_csv("data/clean_data/df_q4.csv")
df_q5 = pd.read_csv("data/clean_data/df_q5.csv")

# Okabe-Ito qualitative palette (orange + blue): colorblind-safe pair.
CHAR1_COLOR = "#E69F00"
CHAR2_COLOR = "#0072B2"
OTHER_COLOR = "#999999"

In [2]: # Helpers etc.
characters_list = (
    df_q1.groupby("character_names")["word_count"]
        .sum()
        .sort_values(ascending=False)
        .index.tolist()
)
seasons_list = sorted(df_q4["season"].unique().tolist())
episodes_list = sorted(df_q4["number_in_season"].unique().tolist())

dropdown_char1 = alt.binding_select(options=characters_list, name="Character 1: ")
char_select_1 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": characters_list[0]}],
    bind=dropdown_char1,
    name="char1",
)

dropdown_char2 = alt.binding_select(options=characters_list, name="Character 2: ")
char_select_2 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": characters_list[1]}], # Default to a different charac
    bind=dropdown_char2,
    name="char2",
)

dropdown_season = alt.binding_select(options=seasons_list, name="Season: ")
season_select = alt.selection_point(
    fields=["season"],
    value=[{"season": seasons_list[0]}],
    bind=dropdown_season,
    name="season_select",
)
```

```

dropdown_episode = alt.binding_select(options=episodes_list, name="Episode: ")
episode_select = alt.selection_point(
    fields=["number_in_season"],
    value=[{"number_in_season": episodes_list[0]}],
    bind=dropdown_episode,
    name="episode_select",
)

# Reusable color encoding: rows matching char_select_1 / char_select_2 take the
# Okabe-Ito orange / blue, everything else stays grey.
pair_color = alt.Color(
    condition=[
        {"param": char_select_1.name, "empty": False, "value": CHAR1_COLOR},
        {"param": char_select_2.name, "empty": False, "value": CHAR2_COLOR},
    ],
    value=OTHER_COLOR,
)

```

Question 1: What are the characters that issue more words, and how are these distributed?

Design Iterations

```

In [3]: # Q1 - iteration 1: table of total word counts per character
words_per_char = df_q1.groupby("character_names", as_index=False)["word_count"].sum()
q1_words_table = (
    words_per_char.rename( # type: ignore[call-overload]
        columns={
            "character_names": "character",
            "word_count": "total_word_count",
        }
    )
    .sort_values("total_word_count", ascending=False)
    .assign(percentage=lambda d: d["total_word_count"] / d["total_word_count"].sum())
    .reset_index(drop=True)
)

q1_words_table.head(13).style.format(
    {"total_word_count": "{:,}.0f}", "percentage": "{:.2%}"})

```

Out [3]:

	character	total_word_count	percentage
0	Homer Simpson	268,457	33.09%
1	Marge Simpson	123,036	15.16%
2	Bart Simpson	109,550	13.50%
3	Lisa Simpson	99,023	12.21%
4	C. Montgomery Burns	35,428	4.37%
5	Moe Szyslak	32,049	3.95%
6	Seymour Skinner	28,723	3.54%
7	Ned Flanders	22,981	2.83%
8	Krusty the Clown	20,503	2.53%
9	Chief Wiggum	20,281	2.50%
10	Grampa Simpson	20,212	2.49%
11	Kent Brockman	15,815	1.95%
12	Milhouse Van Houten	15,265	1.88%

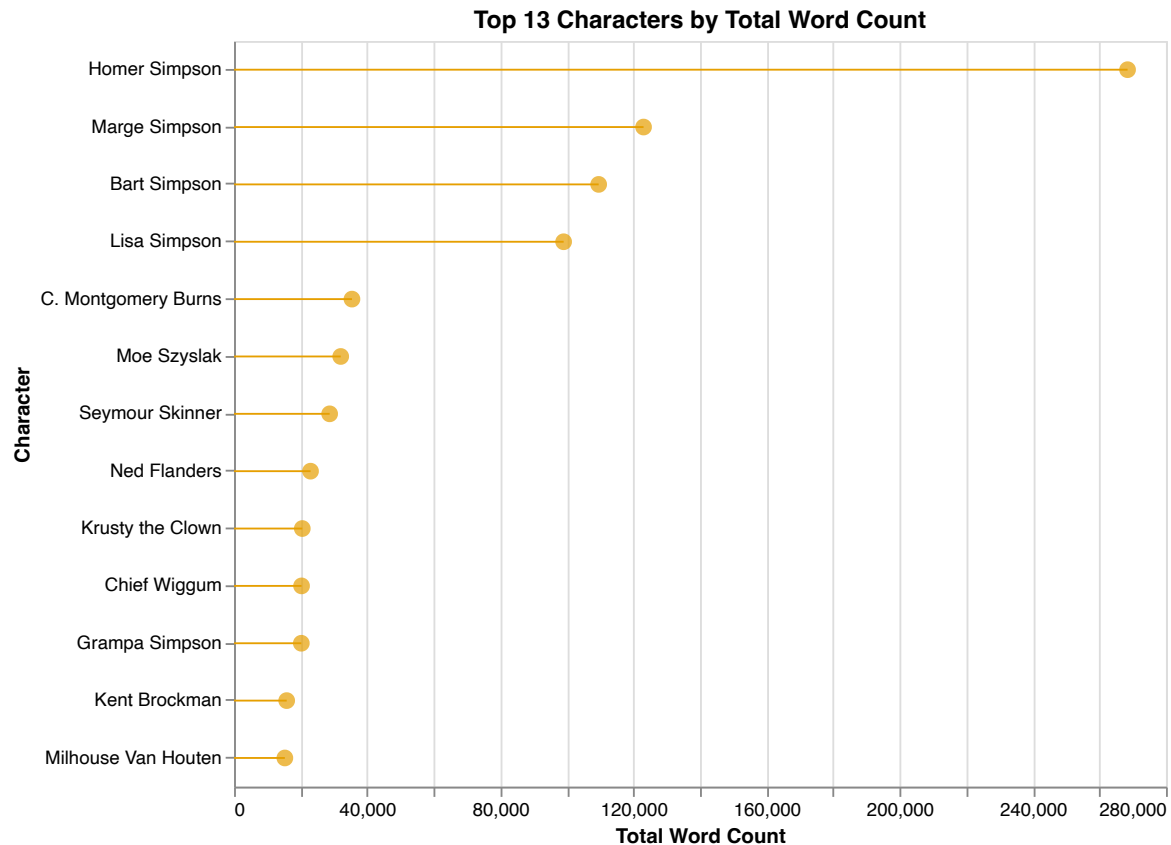
```
In [4]: # Q1 - iteration 2: lollipop chart of top-n characters by total word count
n = 13
top_n_words = (
    df_q1.groupby("character_names", as_index=False)["word_count"]
        .sum()
        .nlargest(n, "word_count")
)

base = alt.Chart(top_n_words).encode(
    x=alt.X("word_count:Q", title="Total Word Count"),
    y=alt.Y("character_names:N", sort="-x", title="Character"),
    tooltip=["character_names", "word_count"],
)

points = base.mark_circle(size=80, color=CHAR1_COLOR)
lines = base.mark_rule(color=CHAR1_COLOR)

(lines + points).properties(
    title=f"Top {n} Characters by Total Word Count", width=500, height=400
)
```

Out [4]:



Final Visualization

```
In [5]: base_chart = alt.Chart(df_q1).transform_joinaggregate(  
    # Total word count per character; reused by both charts to sort.  
    total_count="sum(word_count)",  
    groupby=["character_names"],  
)  
  
bar_chart_word = (  
    base_chart.mark_bar()  
    .encode(  
        y=alt.Y(  
            "character_names:N",  
            title="Character",  
            sort=alt.EncodingSortField(  
                field="total_count", op="max", order="descending"  
            ),  
        ),  
        x=alt.X("sum(word_count):Q", title="Total Word Count"),  
        color=pair_color,  
        tooltip=["character_names", "sum(word_count):Q"],  
    )  
    .add_params(char_select_1, char_select_2)  
    .properties(  
        height=350,  
        width=380,  
        title="Total Word Count per Character",  
    )  
)  
  
boxplot_chart_word = (  
    base_chart.mark_boxplot(size=15)  
    .encode(  
        y=alt.Y(  
            "character_names:N",  
            title="Character",  
            sort=alt.EncodingSortField(  
                field="total_count", op="max", order="descending"
```

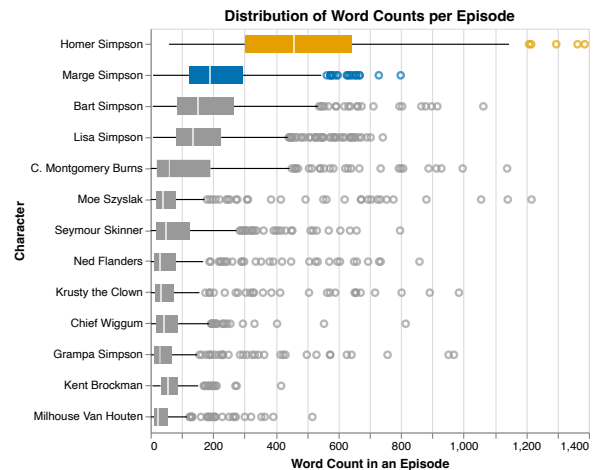
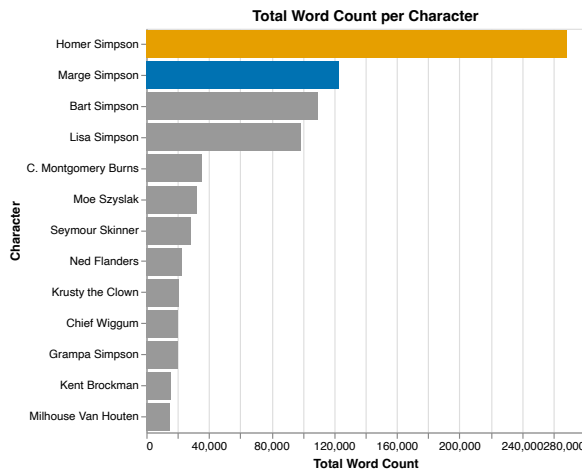
```

),
x=alt.X("word_count:Q", title="Word Count in an Episode"),
color=pair_color,
)
.properties(
  height=350,
  width=380,
  title="Distribution of Word Counts per Episode",
)
)

interactive_dashboard = bar_chart_word | boxplot_chart_word
interactive_dashboard

```

Out [5]:



Character 2: ▼

Character 1: ▼

Design Discussion

We interpreted the first question as having two parts. First, what is the word count per character? Second, how are those word counts distributed across episodes? For the first question we chose a horizontal bar chart sorted by total words issued. A bar chart makes it easy to compare characters, while the horizontal layout allows clear display of character names. Sorting provides a clear ranking from highest to lowest word count. We chose it over a lollipop chart, which suits large numbers of categories, because we limited our final dashboard to the top 13 characters. We chose 13 because it captures the main cast viewers recognize. Beyond that, characters contribute very small word counts, producing a long tail of barely-visible bars that clutters the chart without insight. For the second part, we created a horizontal boxplot, sorted in the same order as the bar chart. This shows how word counts vary across episodes per character, revealing how totals are composed. Two characters are highlighted in orange and blue while the rest stay grey. We applied this focus on a two-character comparison here because later charts centre on it, and carrying it through keeps the design consistent in the final dashboard.

Question 2: How have the word numbers among the characters evolved throughout the available seasons?

Design Iterations

In [6]: `# Q2 – iteration 1: line chart of word count per season for top n characters`
`n = 13`

```

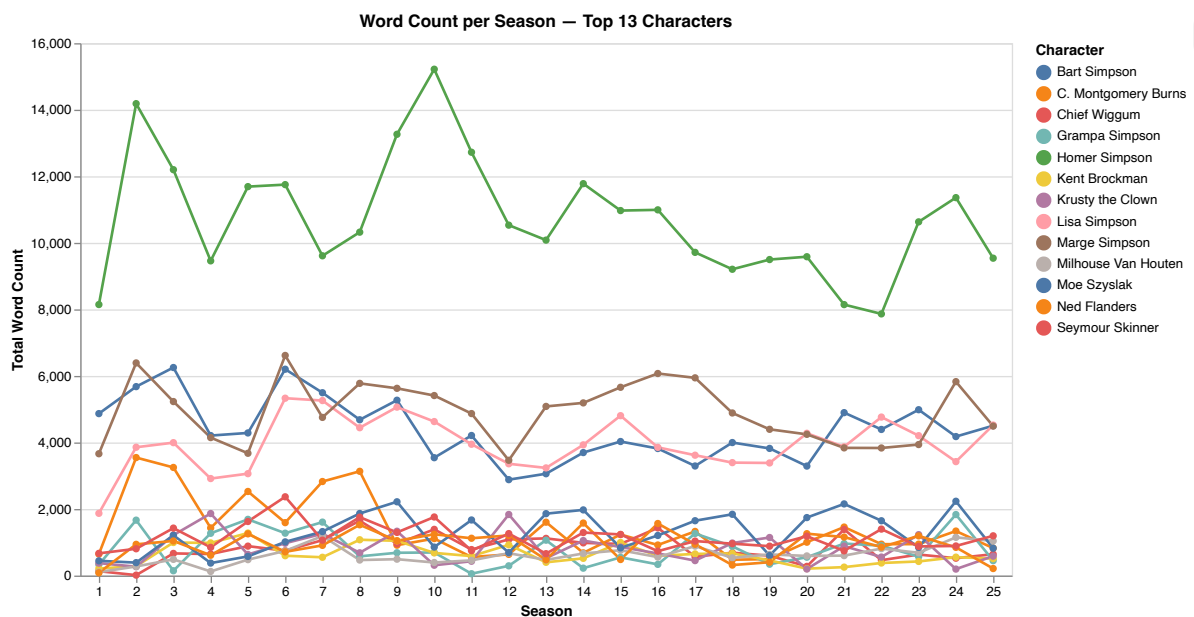
top_characters = (
    df_q2.groupby("character_names")["word_count"].sum().nlargest(n).index.tolist()
)

words_per_season = df_q2[df_q2["character_names"].isin(top_characters)]

alt.Chart(words_per_season).mark_line(point=True).encode(
    x=alt.X("season:Q", title="Season", axis=alt.Axis(labelAngle=0)),
    y=alt.Y("word_count:Q", title="Total Word Count"),
    color=alt.Color("character_names:N", title="Character"),
    tooltip=[
        "character_names",
        "season",
        alt.Tooltip("word_count:Q", format=","),
    ],
).properties(
    title=f"Word Count per Season – Top {n} Characters",
    width=700,
    height=400,
)

```

Out [6]:



In [7]:

```

# Q2 – iteration 2: small multiples line chart per character
n = 13
top_characters = (
    df_q2.groupby("character_names")["word_count"].sum().nlargest(n).index.tolist()
)

words_per_season = df_q2[df_q2["character_names"].isin(top_characters)]

alt.Chart(words_per_season).mark_line(point=True).encode(
    x=alt.X("season:Q", title="Season", axis=alt.Axis(labelAngle=0)),
    y=alt.Y("word_count:Q", title="Word Count"),
    color=alt.Color("character_names:N", legend=None),
).properties(
    width=200,
    height=120,
).facet(
    facet=alt.Facet(
        "character_names:N",
        title=None,
        sort=top_characters,
        header=alt.Header(labelPadding=2, labelFontSize=12),
    ),
    columns=4,
).properties(
    title=alt.TitleParams(

```

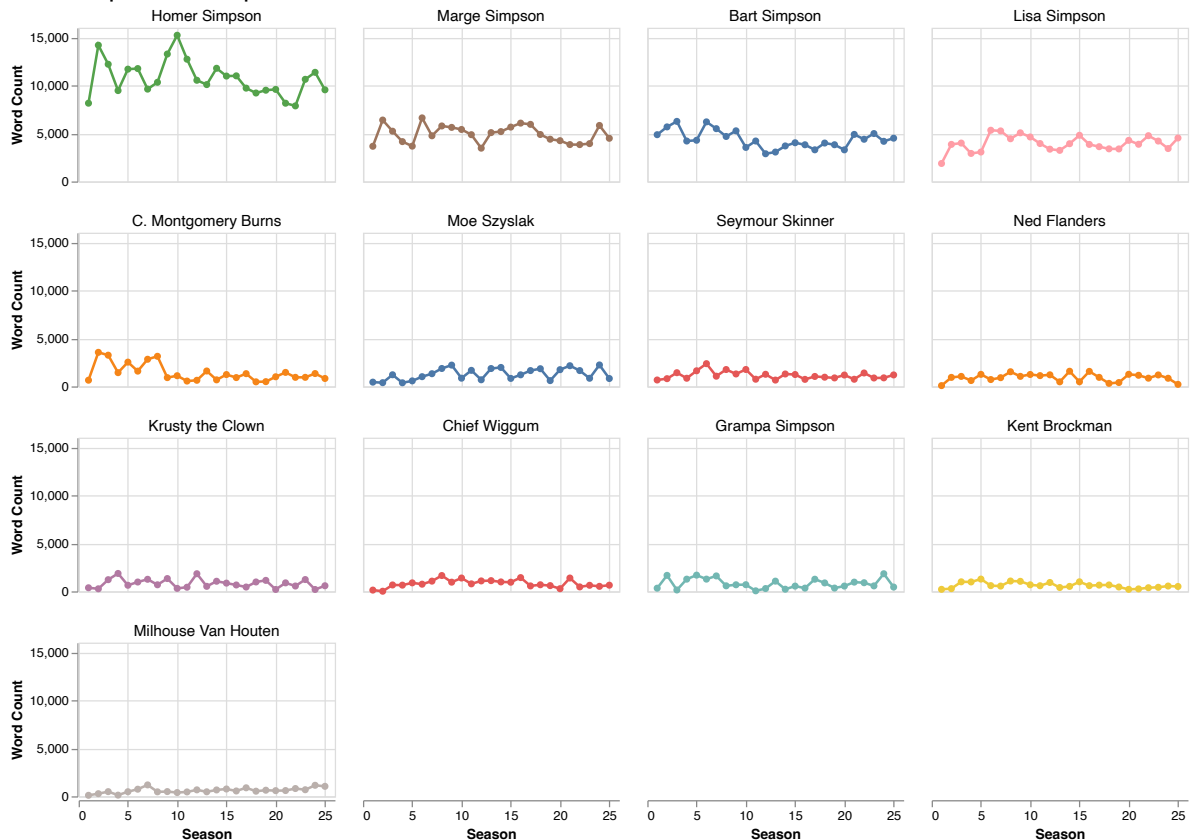
```

        text=f"Word Count per Season – Top {n} Characters",
        anchor="start",
    ),
)

```

Out [7]:

Word Count per Season — Top 13 Characters



Final Visualization

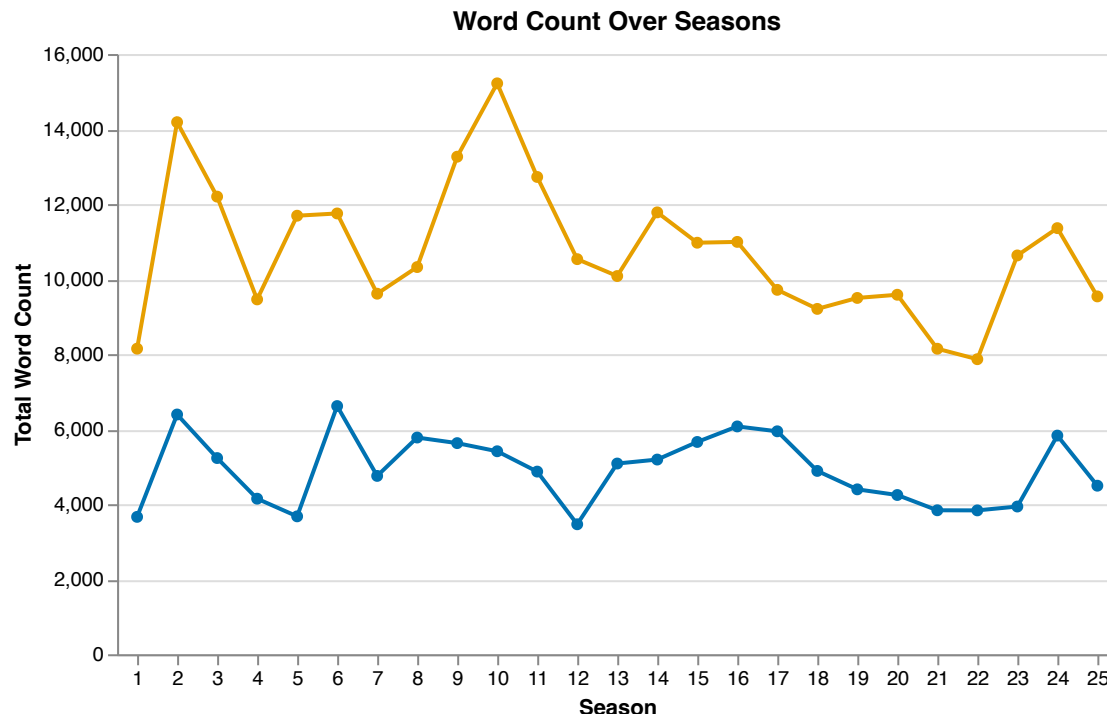
```

In [8]: line_chart = (
    alt.Chart(df_q2)
    .mark_line(
        strokeWidth=2,
        point=alt.OverlayMarkDef(filled=True, size=35),
    )
    .encode(
        x=alt.X("season:0", title="Season", axis=alt.Axis(labelAngle=0)),
        y=alt.Y("word_count:Q", title="Total Word Count"),
        color=pair_color,
        detail="character_names:N",
        tooltip=["character_names", "season", "word_count"],
    )
    .add_params(
        char_select_1,
        char_select_2,
    )
    .transform_filter(char_select_1 | char_select_2)
    .properties(
        title="Word Count Over Seasons",
        height=300,
    )
)

line_chart

```

Out [8]:

Character 2: ▼Character 1: ▼

Design Discussion

To answer how the word counts among the characters evolved throughout the available seasons, we created a multi-line chart, which is well suited to displaying change over time. Our first attempt plotted all 13 characters in a single figure, but this quickly became too cluttered to read. We also tried a multiple-views chart; however, with 13 characters this resulted in too many small plots that were difficult to read and compare effectively.

In the end, we decided to carry the two-character focus of questions 3 and 4 over to this chart as well, showing only the two selected characters. This keeps the chart readable while allowing a clear comparison of trends, and it keeps the two-character focus consistent across the final dashboard.

Question 3: Compare the word distribution for a pair of characters, for a selected season

Design Iterations

```
In [9]: # Q3 - iteration 1: grouped bar chart - word count per episode, two characters side b
q3i1_char1 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Homer Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 1: "),
    name="q3i1_char1",
)
q3i1_char2 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Marge Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 2: "),
    name="q3i1_char2",
)
q3i1_season = alt.selection_point(
    fields=["season"],
```



```

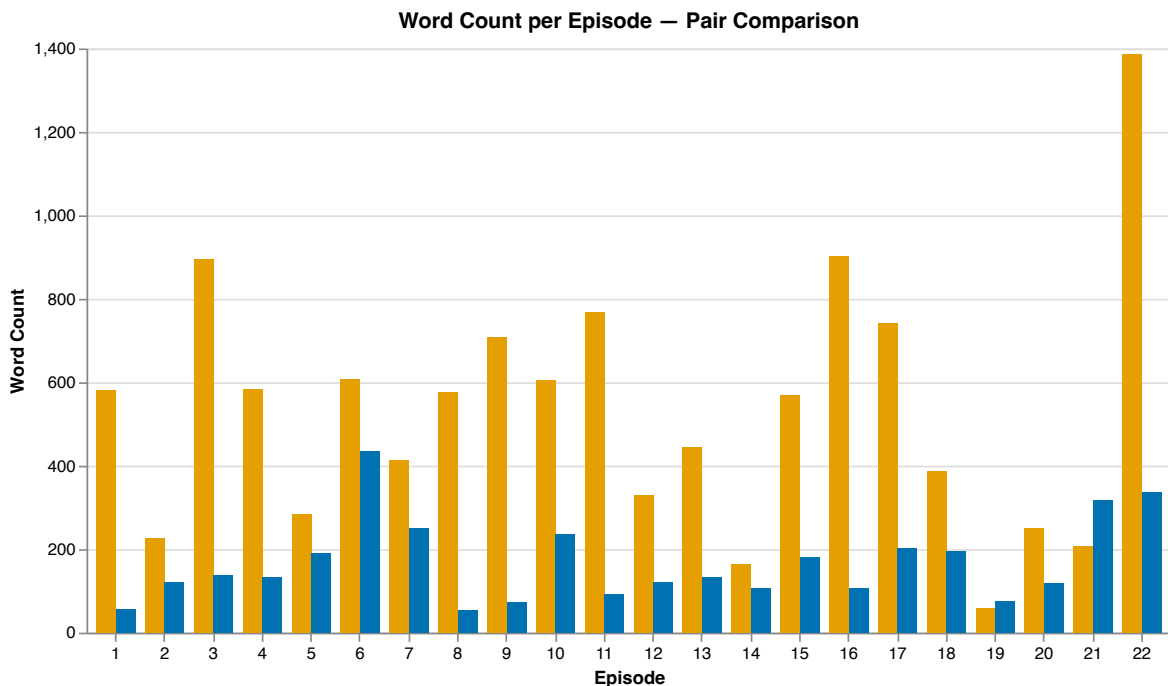
value=[{"season": 5}],
bind=alt.binding_select(options=seasons_list, name="Season: "),
name="q3i1_season",
)

q3i1_color = alt.Color(
    condition=[
        {"param": q3i1_char1.name, "empty": False, "value": CHAR1_COLOR},
        {"param": q3i1_char2.name, "empty": False, "value": CHAR2_COLOR},
    ],
    value=OTHER_COLOR,
)

alt.Chart(df_q3).mark_bar().encode(
    x=alt.X("number_in_season:0", title="Episode", axis=alt.Axis(labelAngle=0)),
    xOffset=alt.XOffset("character_names:N"),
    y=alt.Y("word_count:Q", title="Word Count"),
    color=q3i1_color,
    tooltip=[
        "character_names",
        alt.Tooltip("number_in_season:0", title="Episode"),
        alt.Tooltip("word_count:Q", format=","),
    ],
).add_params(q3i1_char1, q3i1_char2, q3i1_season).transform_filter(
    q3i1_season & (q3i1_char1 | q3i1_char2)
).properties(
    title="Word Count per Episode – Pair Comparison",
    width=650,
    height=350,
)

```

Out [9]:



Season: 5 ▼

Character 2: Marge Simpson ▼

Character 1: Homer Simpson ▼

```

In [10]: # Q3 - iteration 2: normalized stacked bar chart – proportion per episode
q3i2_char1 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Homer Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 1: "),
    name="q3i2_char1",
)

q3i2_char2 = alt.selection_point(
    fields=["character_names"],

```

```

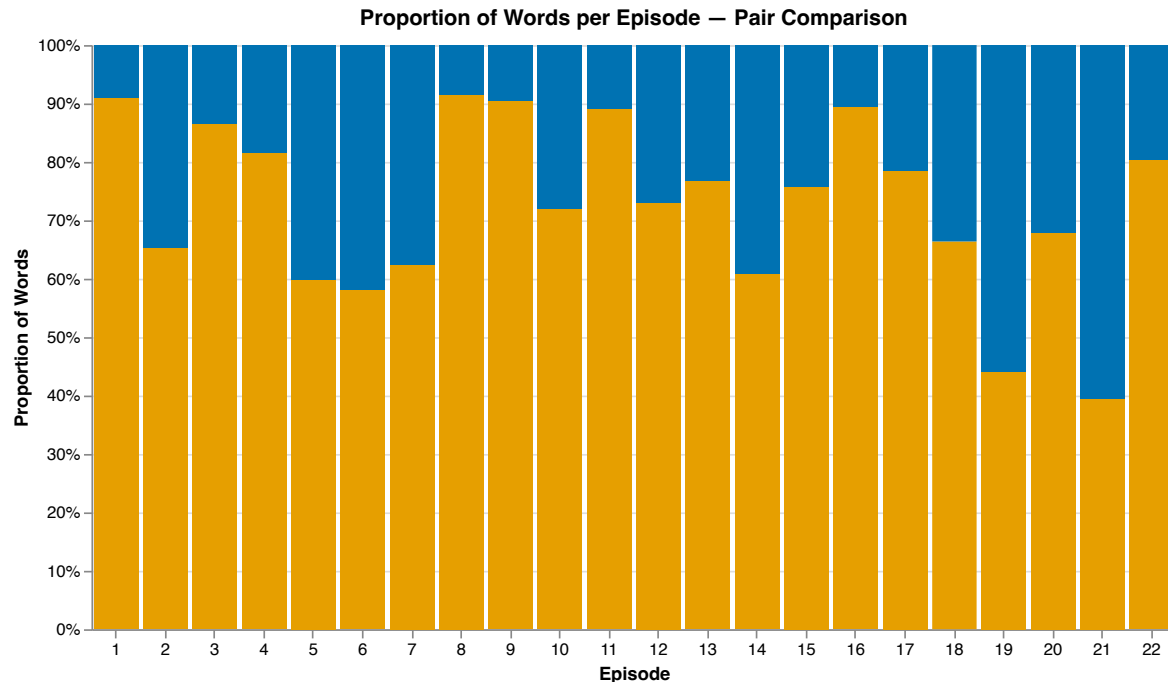
        value=[{"character_names": "Marge Simpson"}],
        bind=alt.binding_select(options=characters_list, name="Character 2: "),
        name="q3i2_char2",
    )
    q3i2_season = alt.selection_point(
        fields=["season"],
        value=[{"season": 5}],
        bind=alt.binding_select(options=seasons_list, name="Season: "),
        name="q3i2_season",
    )

    q3i2_color = alt.Color(
        condition=[
            {"param": q3i2_char1.name, "empty": False, "value": CHAR1_COLOR},
            {"param": q3i2_char2.name, "empty": False, "value": CHAR2_COLOR},
        ],
        value=OTHER_COLOR,
    )

    alt.Chart(df_q3).mark_bar().encode(
        x=alt.X("number_in_season:0", title="Episode", axis=alt.Axis(labelAngle=0)),
        y=alt.Y(
            "word_count:Q",
            title="Proportion of Words",
            stack="normalize",
            axis=alt.Axis(format="%"),
        ),
        color=q3i2_color,
        order=alt.Order("character_names:N"),
        tooltip=[
            "character_names",
            alt.Tooltip("number_in_season:0", title="Episode"),
            alt.Tooltip("word_count:Q", format=","), title="Words"),
        ],
    ).add_params(q3i2_char1, q3i2_char2, q3i2_season).transform_filter(
        q3i2_season & (q3i2_char1 | q3i2_char2)
    ).properties(
        title="Proportion of Words per Episode – Pair Comparison",
        width=650,
        height=350,
    )

```

Out [10]:



Season: 5

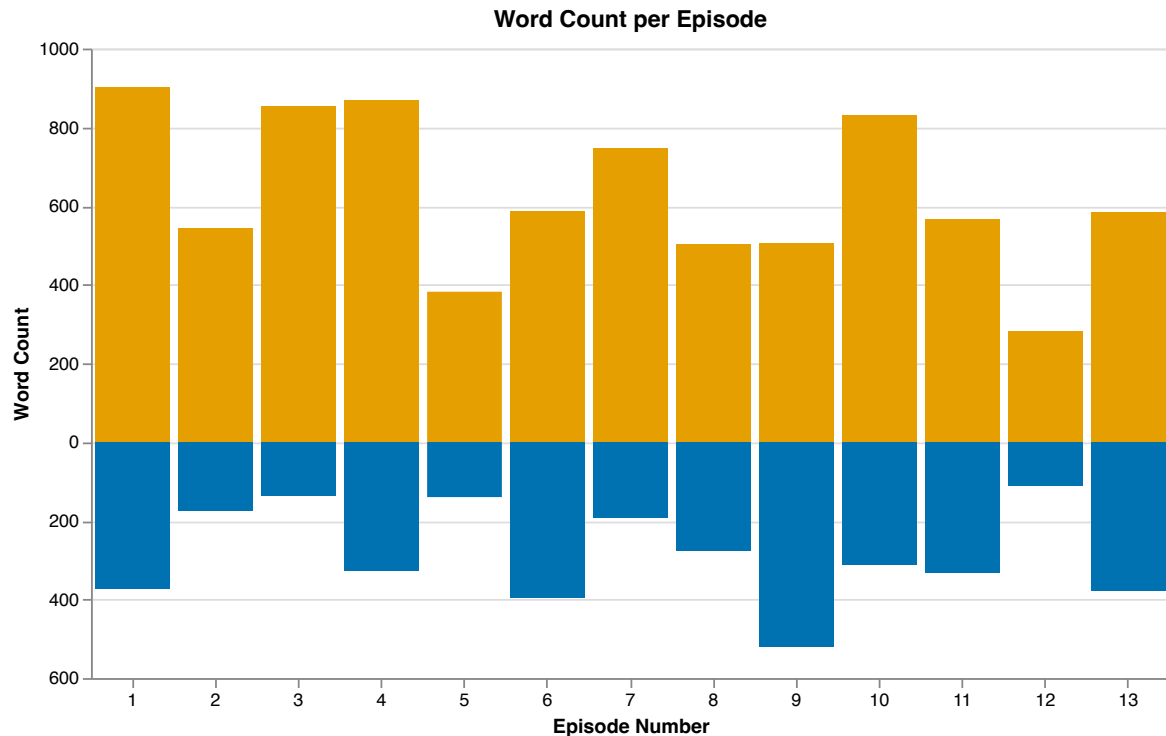
Character 2: Marge Simpson

Character 1: Homer Simpson

Final Visualization

```
In [11]: butterfly_chart = (  
    alt.Chart(df_q3)  
    .mark_bar()  
    .encode(  
        x=alt.X(  
            "number_in_season:0",  
            title="Episode Number",  
            axis=alt.Axis(labelAngle=0),  
        ),  
        y=alt.Y(  
            "signed_word_count:Q",  
            title="Word Count",  
            axis=alt.Axis(labelExpr="abs(datum.value)"),  
        ),  
        color=pair_color,  
        tooltip=["character_names", "season", "number_in_season", "word_count"],  
    )  
    .add_params(  
        char_select_1,  
        char_select_2,  
        season_select,  
    )  
    .transform_filter(season_select & (char_select_1 | char_select_2))  
    .transform_calculate(  
        signed_word_count=f"datum.character_names == {char_select_2.name}.character_n  
    )  
    .properties(  
        title="Word Count per Episode",  
        height=350,  
        width=600,  
    )  
)  
  
butterfly_chart
```

Out [11]:



Season: 1

Character 2: Marge Simpson

Character 1: Homer Simpson

Design Discussion

To show a season's word distribution for a pair of concrete characters, we created a vertical version of a butterfly chart. Each episode of the selected season is placed along the horizontal axis, with the first character's word count drawn as bars extending upward from a shared baseline and the second character's extending downward. The vertical axis is relabeled to show absolute values so that both directions read as positive counts, while a single shared linear scale ensures that bar length is proportional to the word count and directly comparable between the two characters.

One strong alternative we considered was a grouped bar chart, but we decided against it because it would double the number of bars and interleave the two characters, making it harder to follow each character's distribution across the season. Our final version gives each character its own half-plane, keeping their per-episode patterns easy to read. Here a drawback is that opposing bars of similar length are hard to compare precisely. We considered a visual marker to highlight the larger one but decided against it to avoid visual noise, since our focus is the general distribution and exact values remain available on hover as details on demand.

Question 4: Compare the word distribution for a pair of characters, over a concrete episode

Design Iterations

```
In [12]: # Q4 - iteration 3: binned normalized stacked bar - proportion per episode segment
q4i3_char1 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Homer Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 1: "),
    name="q4i3_char1",
)
```

```

q4i3_char2 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Marge Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 2: "),
    name="q4i3_char2",
)
q4i3_season = alt.selection_point(
    fields=["season"],
    value=[{"season": 5}],
    bind=alt.binding_select(options=seasons_list, name="Season: "),
    name="q4i3_season",
)
q4i3_episode = alt.selection_point(
    fields=["number_in_season"],
    value=[{"number_in_season": 1}],
    bind=alt.binding_select(options=episodes_list, name="Episode: "),
    name="q4i3_episode",
)

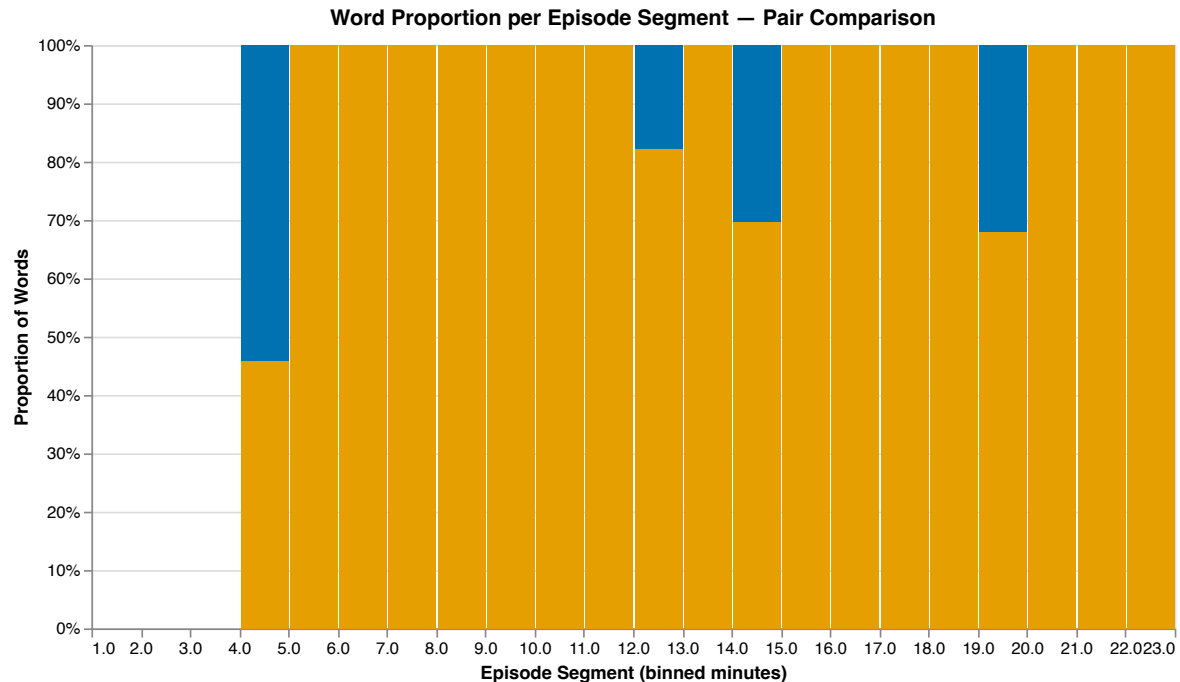
q4i3_color = alt.Color(
    condition=[
        {"param": q4i3_char1.name, "empty": False, "value": CHAR1_COLOR},
        {"param": q4i3_char2.name, "empty": False, "value": CHAR2_COLOR},
    ],
    value=OTHER_COLOR,
)

# Bin episode minutes into ~10 segments so each bar represents a portion of the episode
n_bins = 23

alt.Chart(df_q4).mark_bar().encode(
    x=alt.X(
        "timestamp_in_min:Q",
        bin=alt.Bin(maxbins=n_bins),
        title="Episode Segment (binned minutes)",
        axis=alt.Axis(labelAngle=0),
    ),
    y=alt.Y(
        "word_count:Q",
        title="Proportion of Words",
        stack="normalize",
        axis=alt.Axis(format="%"),
    ),
    color=q4i3_color,
    order=alt.Order("character_names:N"),
    tooltip=[
        "character_names",
        alt.Tooltip("timestamp_in_min:Q", title="Minute"),
        alt.Tooltip("word_count:Q", format=","), title="Words"),
    ],
).add_params(q4i3_char1, q4i3_char2, q4i3_season, q4i3_episode).transform_filter(
    q4i3_season & q4i3_episode & (q4i3_char1 | q4i3_char2)
).properties(
    title="Word Proportion per Episode Segment – Pair Comparison",
    width=650,
    height=350,
)

```

Out [12]:



Episode: 1

Season: 5

Character 2: Marge Simpson

Character 1: Homer Simpson

In [13]: # Q4 - iteration 4: grouped bar chart - word count per minute, two characters side by

```
q4i4_char1 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Homer Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 1: "),
    name="q4i4_char1",
)
q4i4_char2 = alt.selection_point(
    fields=["character_names"],
    value=[{"character_names": "Marge Simpson"}],
    bind=alt.binding_select(options=characters_list, name="Character 2: "),
    name="q4i4_char2",
)
q4i4_season = alt.selection_point(
    fields=["season"],
    value=[{"season": 5}],
    bind=alt.binding_select(options=seasons_list, name="Season: "),
    name="q4i4_season",
)
q4i4_episode = alt.selection_point(
    fields=["number_in_season"],
    value=[{"number_in_season": 1}],
    bind=alt.binding_select(options=episodes_list, name="Episode: "),
    name="q4i4_episode",
)
q4i4_color = alt.Color(
    condition=[
        {"param": q4i4_char1.name, "empty": False, "value": CHAR1_COLOR},
        {"param": q4i4_char2.name, "empty": False, "value": CHAR2_COLOR},
    ],
    value=OTHER_COLOR,
)

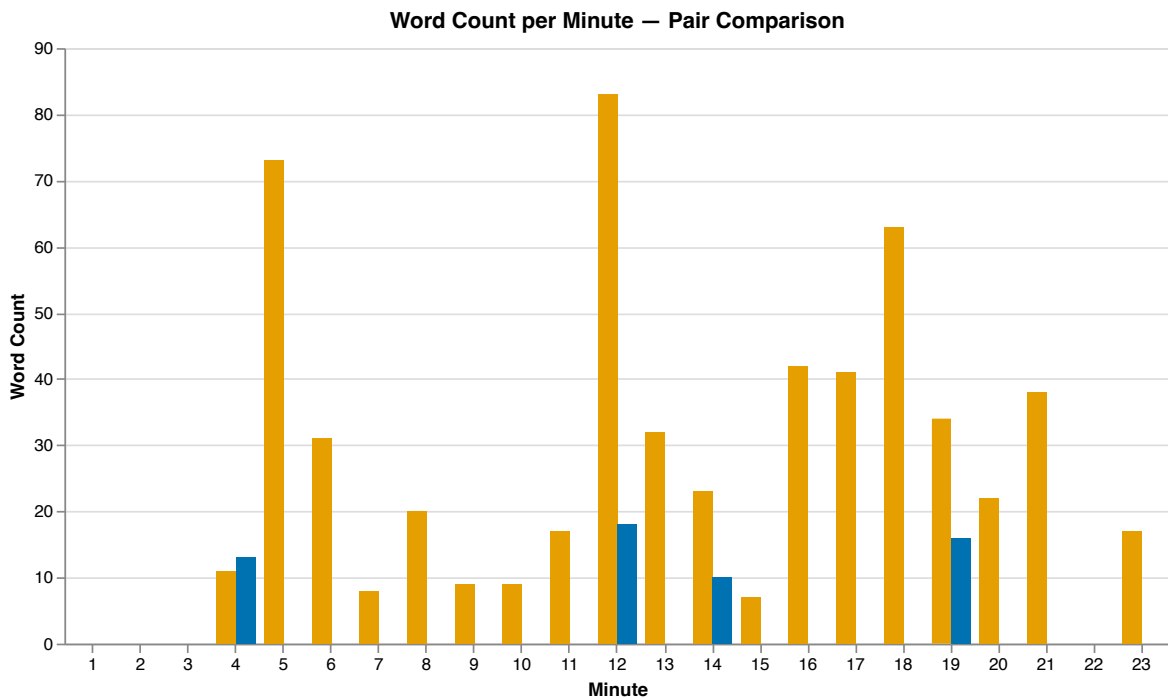
alt.Chart(df_q4).mark_bar().encode(
    x=alt.X("timestamp_in_min:O", title="Minute", axis=alt.Axis(labelAngle=0)),
    xOffset=alt.XOffset("character_names:N"),
    y=alt.Y("word_count:Q", title="Word Count"),
    color=q4i4_color,
```

```

        tooltip=[
            "character_names",
            alt.Tooltip("timestamp_in_min:0", title="Minute"),
            alt.Tooltip("word_count:Q", format=","),
        ],
    ).add_params(q4i4_char1, q4i4_char2, q4i4_season, q4i4_episode).transform_filter(
        q4i4_season & q4i4_episode & (q4i4_char1 | q4i4_char2)
    ).properties(
        title="Word Count per Minute – Pair Comparison",
        width=650,
        height=350,
    )

```

Out [13]:



Episode: 1 ▼

Season: 5 ▼

Character 2: Marge Simpson ▼

Character 1: Homer Simpson ▼

Final Visualization

```

In [14]: butterfly_chart = (
    alt.Chart(df_q4)
    .mark_bar()
    .encode(
        x=alt.X(
            "timestamp_in_min:0",
            title="Minute of Episode",
            axis=alt.Axis(labelAngle=0, labelOverlap=True),
        ),
        y=alt.Y(
            "signed_word_count:Q",
            title="Word Count",
            axis=alt.Axis(labelExpr="abs(datum.value)"),
        ),
        color=pair_color,
        tooltip=[
            "character_names",
            "season",
            "number_in_season",
            "timestamp_in_min",
            "word_count",
        ],
    )

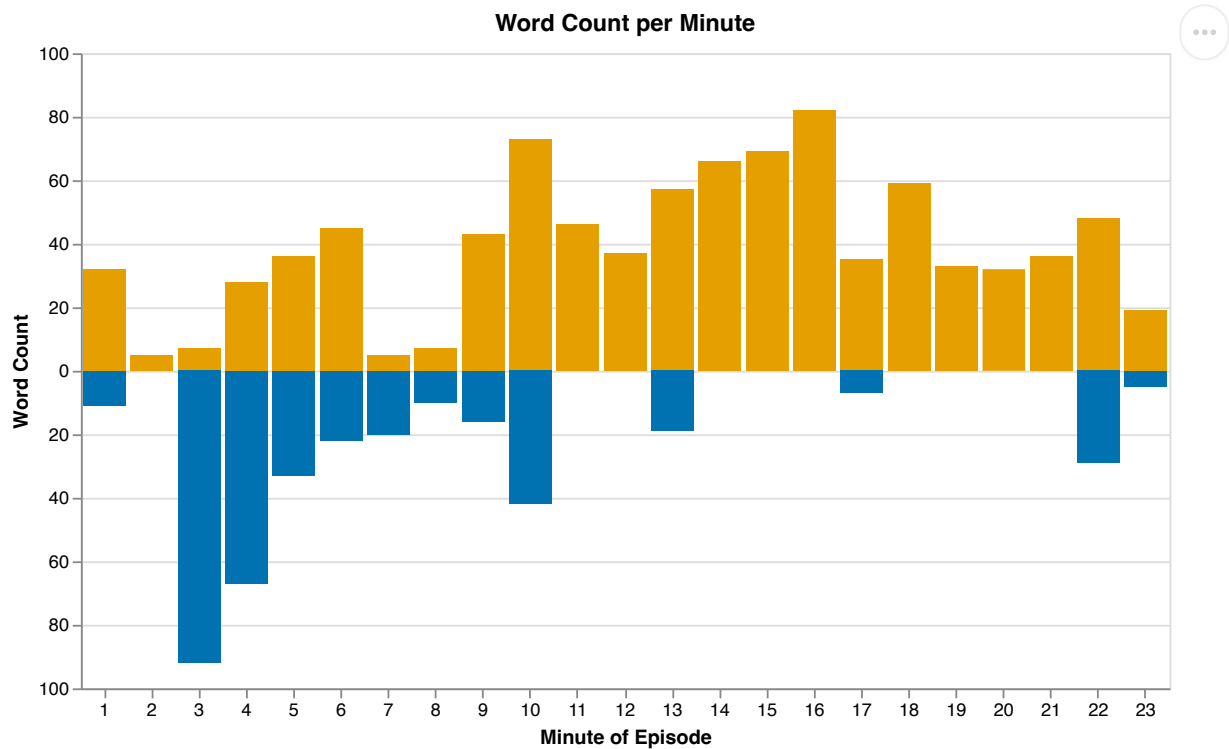
```

```

.add_params(
    char_select_1,
    char_select_2,
    season_select,
    episode_select,
)
.transform_filter(season_select & episode_select & (char_select_1 | char_select_2
.transform_calculate(
    signed_word_count=f"datum.character_names == {char_select_2.name}.character_n
)
)
)
.properties(
    title="Word Count per Minute",
    height=350,
    width=600,
)
)
butterfly_chart

```

Out[14]:



Episode: 1 ▼
Season: 1 ▼
Character 2: Marge Simpson ▼
Character 1: Homer Simpson ▼

Design Discussion

Since question 4 asks for the same comparison as question 3 but within a single episode, we reused the vertical butterfly chart and its entire design. Instead of episodes, the horizontal axis now shows each minute of the selected episode, while the vertical axis shows the summed word count for that minute. The underlying data records individual words at exact timestamps. In theory this would allow us to plot them directly, but they would form a sparse stream of events with no easily readable shape, so we aggregate them into time intervals to reveal the actual distribution of speech across the episode. We experimented with different interval sizes and settled on one-minute bins, as they offer a good middle ground between detail and clarity. By default, Altair only draws bars for minutes with data and drops empty ones. We kept the empty minutes visible without a bar so the axis stays consistent across episodes and gaps where a character is silent remain clearly visible. We also tried other iterations but ultimately kept the same design as question 3, so the reader only needs to understand one chart type for both questions rather than learning a new one.

Question 5: What are the characters that issue more sentences, and how are the sentence numbers distributed?

Design Iterations

```
In [15]: # Q5 - iteration 1
sentences_per_char = df_q5.groupby("character_names", as_index=False)[
    "sentence_count"
].sum()
q5_sentences_table = (
    sentences_per_char.rename( # type: ignore[call-overload]
        columns={
            "character_names": "character",
            "sentence_count": "total_sentence_count",
        }
    )
    .sort_values("total_sentence_count", ascending=False)
    .assign(
        percentage=lambda d: d["total_sentence_count"] / d["total_sentence_count"].sum()
    )
    .reset_index(drop=True)
)

print(f"Overall length of q5_sentences_table: {len(q5_sentences_table)}")
```

Overall length of q5_sentences_table: 13

```
In [16]: q5_sentences_table.head(15).style.format(
    {"total_sentence_count": "{:, .0f}", "percentage": "{:.2%}"})
)
```

Out [16]:

	character	total_sentence_count	percentage
0	Homer Simpson	49,407	34.89%
1	Bart Simpson	20,684	14.61%
2	Marge Simpson	20,500	14.48%
3	Lisa Simpson	16,995	12.00%
4	C. Montgomery Burns	5,867	4.14%
5	Moe Szyslak	5,412	3.82%
6	Seymour Skinner	4,461	3.15%
7	Ned Flanders	3,579	2.53%
8	Chief Wiggum	3,449	2.44%
9	Krusty the Clown	3,373	2.38%
10	Grampa Simpson	3,314	2.34%
11	Milhouse Van Houten	2,874	2.03%
12	Kent Brockman	1,695	1.20%

```
In [17]: q5_sentences_table.tail().style.format(
        {"total_sentence_count": "{:,.0f}", "percentage": "{:.2%}"})
```

Out [17]:

	character	total_sentence_count	percentage
8	Chief Wiggum	3,449	2.44%
9	Krusty the Clown	3,373	2.38%
10	Grampa Simpson	3,314	2.34%
11	Milhouse Van Houten	2,874	2.03%
12	Kent Brockman	1,695	1.20%

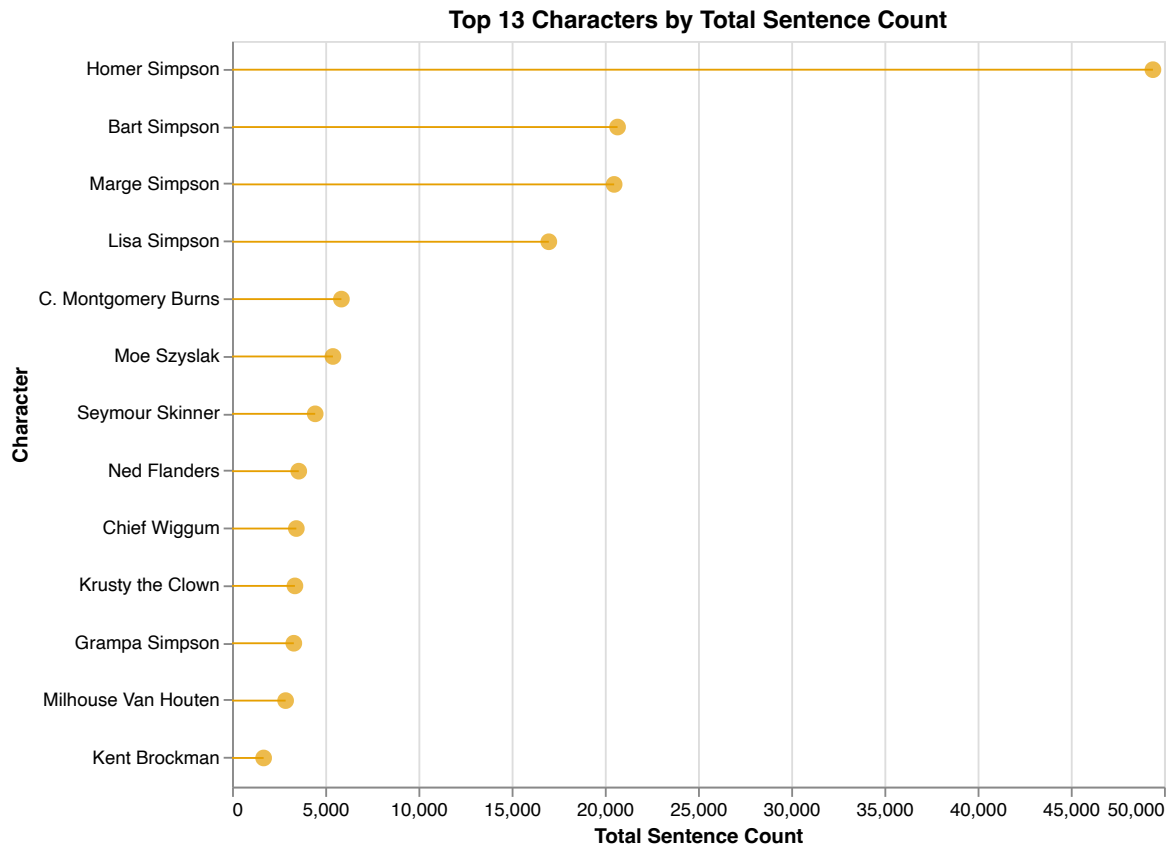
```
In [18]: # Q5 - iteration 3: lollipop chart of top-n characters by total sentence count
n = 13
top_n_sentences = (
    df_q5.groupby("character_names", as_index=False)["sentence_count"]
        .sum()
        .nlargest(n, "sentence_count")
)

base = alt.Chart(top_n_sentences).encode(
    x=alt.X("sentence_count:Q", title="Total Sentence Count"),
    y=alt.Y("character_names:N", sort="-x", title="Character"),
    tooltip=["character_names", "sentence_count"],
)

points = base.mark_circle(size=80, color=CHAR1_COLOR)
lines = base.mark_rule(color=CHAR1_COLOR)

(lines + points).properties(
    title=f"Top {n} Characters by Total Sentence Count", width=500, height=400
)
```

Out [18]:



Final Visualization

```
In [19]: base_chart = alt.Chart(df_q5).transform_joinaggregate(  
    # Total sentence count per character; reused by both charts to sort.  
    total_count="sum(sentence_count)",  
    groupby=["character_names"],  
)  
  
bar_chart_sentence = (  
    base_chart.mark_bar()  
    .encode(  
        y=alt.Y(  
            "character_names:N",  
            title="Character",  
            sort=alt.EncodingSortField(  
                field="total_count", op="max", order="descending"  
            ),  
        ),  
        x=alt.X("sum(sentence_count):Q", title="Total Sentence Count"),  
        color=pair_color,  
        tooltip=["character_names", "sum(sentence_count):Q"],  
    )  
    .add_params(char_select_1, char_select_2)  
    .properties(  
        height=350,  
        width=380,  
        title="Total Sentence Count per Character",  
    )  
)  
  
boxplot_chart_sentence = (  
    base_chart.mark_boxplot(size=15)  
    .encode(  
        y=alt.Y(  
            "character_names:N",  
            title="Character",  
            sort=alt.EncodingSortField(  
                field="total_count", op="max", order="descending"
```

```

),
x=alt.X("sentence_count:Q", title="Sentence Count in an Episode"),
color=pair_color,
)
.properties(
height=350,
width=380,
title="Distribution of Sentence Counts per Episode",
)
)

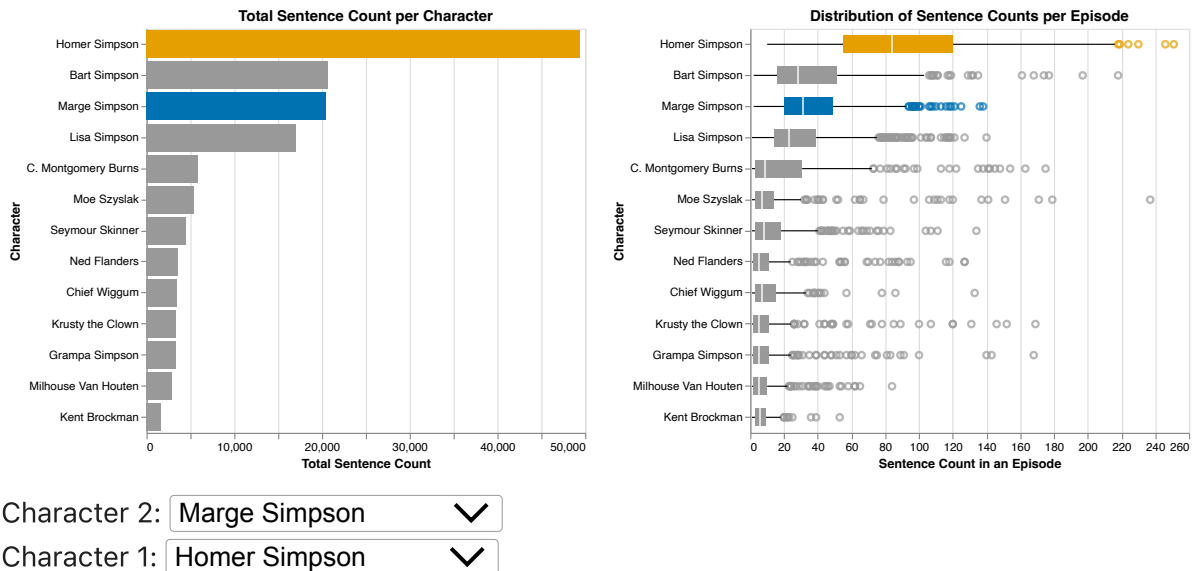
```

```

interactive_dashboard = bar_chart_sentence | boxplot_chart_sentence
interactive_dashboard

```

Out[19]:



Design Discussion

Since question 5 mirrors question 1, we used the same design, now showing total sentence counts. We again chose a horizontal bar chart, now sorted by total sentences issued, for the same reasons as before: the bars make characters easy to compare, the horizontal layout displays their names clearly, and sorting creates a clear ranking. As in question 1, we kept the top 13 characters and favoured a bar chart over a lollipop chart, since the lollipop's advantage only appears with larger numbers of categories. For the distribution we again used a horizontal boxplot, sorted in the same order, showing how each character's sentence counts vary across episodes. The two selected characters stay highlighted in orange and blue for consistency. These colors come from the colorblind-safe Okabe-Ito palette, which we applied consistently across all charts so the two selected characters are always identifiable.

Final Multi-View Visualization

Dashboard Controls

Metric:

Word Count

Sentence Count

Character 1:

Homer Simpson

▼

Character 2:

Marge Simpson

▼

Season:

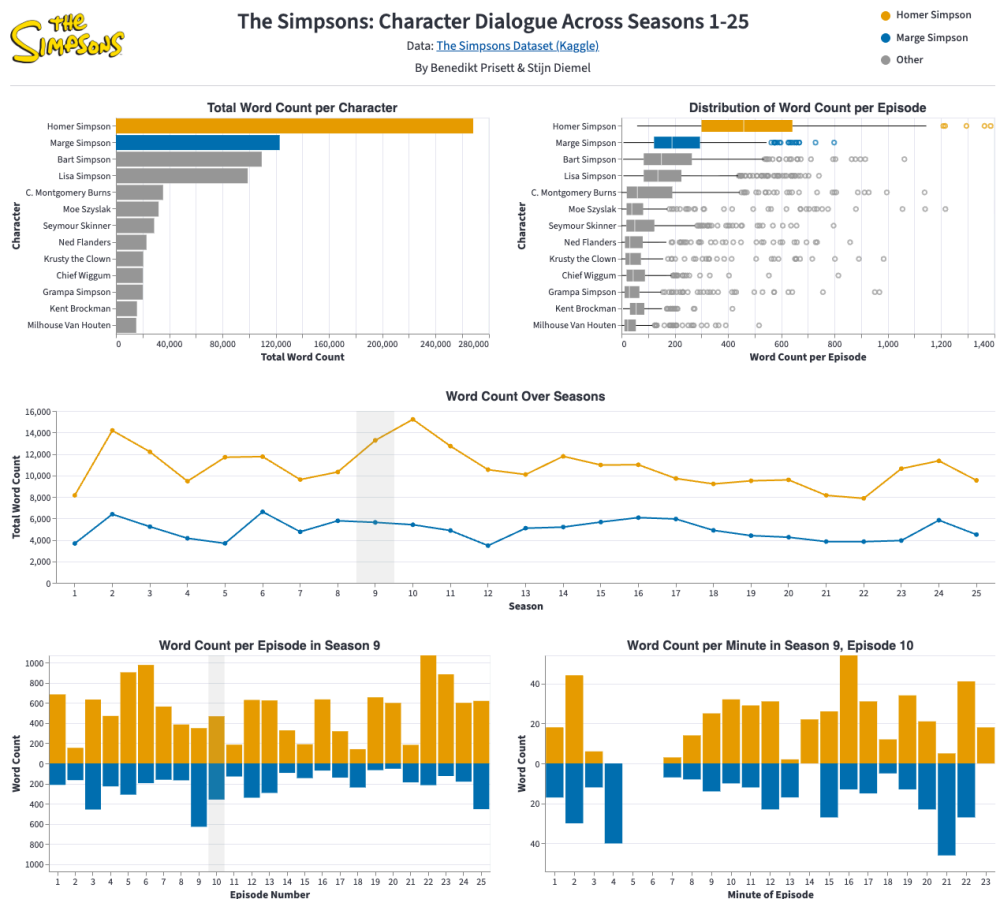
9

▼

Episode:

10

▼



Design Discussion

The final visualization combines all charts into an interactive multi-view dashboard. The charts are arranged so the reader starts with the high-level question of series totals at the top, then progressively drills down to finer granularities (season, episode, minute level).

The main interaction happens through the side panel. Instead of showing both charts for questions 1 and 5, a toggle switches between word and sentence count. We applied this globally rather than only to the bar chart, keeping the whole dashboard on a single metric at a time. This avoids confusion and lets the reader compare patterns consistently across granularities.

The panel also supports further cross-interaction, letting the user select the two characters to compare and choose a season and episode. For the season and episode selection, the user can also click directly on the charts for questions 2 and 3, with the current selection marked by a grey highlight. This makes drilling down quicker. We kept the dropdowns alongside it for consistency and because clicking might not be obvious to all readers. We did not add click selection for the characters, as a single click cannot indicate which of the two slots it should fill; dropdowns avoid this ambiguity.